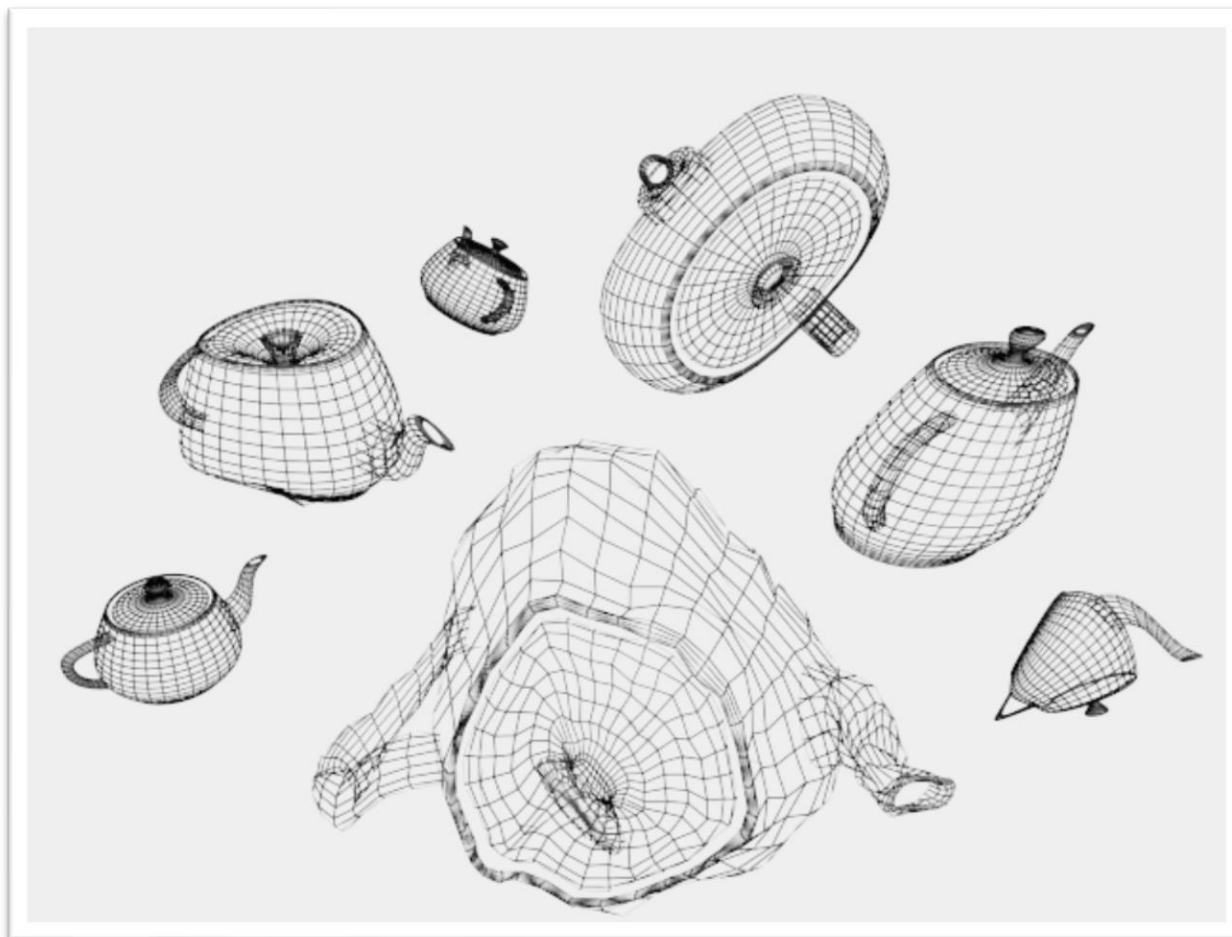
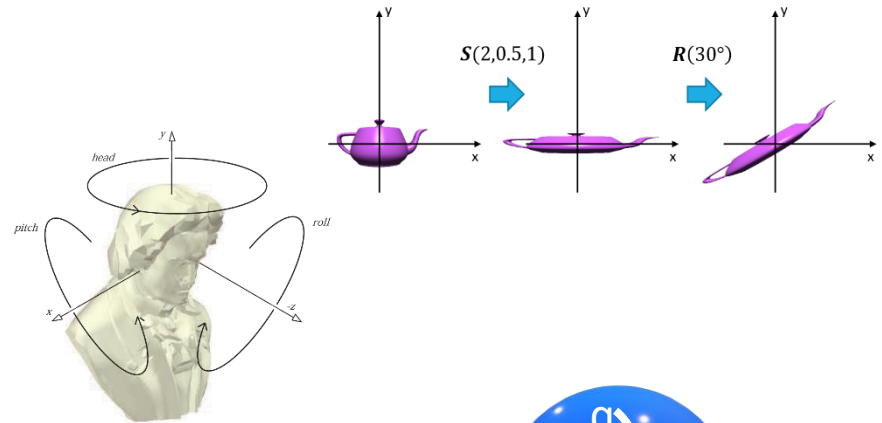


Transformări



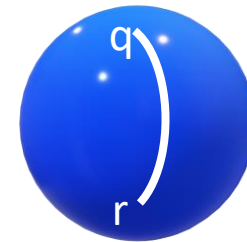
Cuprins

Transformări geometrice de bază

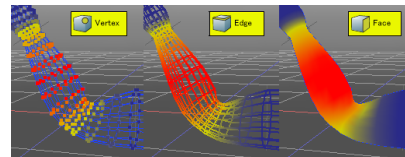


Transformări geometrice speciale

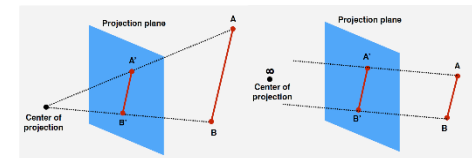
Cuaternioni



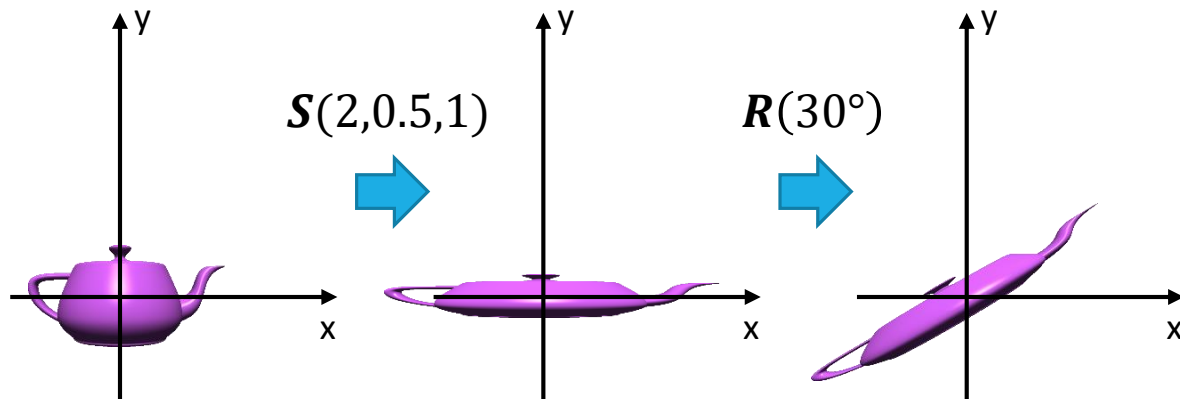
Vertex blending și morfism



Proiecții



Transformări geometrice de bază



Transformări geometrice

Operații care transformă/convertesc entități

- Puncte
- Vectori
- Culori

Poziționare, remodelare sau animare

- Obiecte
- Surse de lumină
- Camere de vizualizare

Asigurarea efectuării calculelor în același spațiu de coordonate

- Ex. Spațiul scenă sau spațiul de vizualizare

Proiecția obiectelor pe un plan

Transformări liniare

O funcție $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ este o transformare liniară dacă:

$$T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}), \forall \mathbf{u}, \mathbf{v} \in \mathbb{R}^n$$

$$T(c\mathbf{v}) = cT(\mathbf{v}), \forall \mathbf{v} \in \mathbb{R}^n, \forall c \in \mathbb{R}$$

Sau

$$T(c_1\mathbf{u} + c_2\mathbf{v}) = c_1T(\mathbf{u}) + c_2T(\mathbf{v}), \forall \mathbf{u}, \mathbf{v} \in \mathbb{R}^n, \forall c_1, c_2 \in \mathbb{R}$$

Pot fi reprezentate matriceal

- Matrici 3×3

Scalarea și rotația – transformări liniare

- Pot fi combinate într-o singură transformare liniară – matrice 3×3

Transformări afine

Translația **nu este o transformare liniară**

Translația – transformare afină

Forma matriceală

- Matrici 4×4

Transformare liniară



Scalări + rotații + translații – transformare afină

- Pot fi combinate într-o singură transformare afină
– matrice 4×4

Coordonate omogene

Reprezentarea similară a vectorilor direcție și a punctelor

Coordonată suplimentară – w

- Vectori direcție – $v = (v_x, v_y, v_z, 0)^T, w = 0$
- Puncte – $v = (v_x, v_y, v_z, 1)^T, w = 1$

Toate matricile de translație, rotație, scalare, reflexie, shear (forfecare) – matrici afine

- Se aplică la fel vectorilor direcție și punctelor

Transformările afine

- Păstrează paralelismul liniilor
- Nu păstrează neapărat lungimile și unghiurile

Transformări geometrice

Notăție	Nume	Caracteristici
$T(t)$	Matrice de translație	Mută un punct. Afină.
$R_x(\rho)$	Matrice de rotație	Rotește cu ρ radiani în jurul axei x . Notății similare pentru axele y și z . Ortogonală și afină.
R	Matrice de rotație	Orice matrice de rotație. Ortogonală și afină.
$S(s)$	Matrice de scalare	Scalează de-a lungul axelor x , y și z conform vectorului s . Afină.

Transformări geometrice

Notăție	Nume	Caracteristici
$H_{ij}(s)$	Matrice de shear (forfecare)	Realizează forfecarea componentei i cu un factor s față de componenta j . $i, j \in \{x, y, z\}$. Afină.
$E(h, p, r)$	Transformare Euler	Matrice de orientare dată de unghiurile lui Euler – head (yaw), pitch, roll. Ortogonală și afină.
$P_o(s)$	Proiecție ortogonală	Proiectează paralel pe un plan sau într-un volum. Afină.

Transformări geometrice

Notăție	Nume	Caracteristici
$P_p(s)$	Proiecție perspectivă	Proiectează cu perspectivă pe un plan sau într-un volum.
$slerp(\hat{q}, \hat{r}, t)$	Transformarea slerp	Creează un cuaternion interpolat între cuaternionii $\hat{q}$ și $\hat{r}$ și parametrul t

Translația

Schimbarea unei locații în alta

Deplasează o entitate după un vector $\mathbf{t} = (t_x, t_y, t_z)$

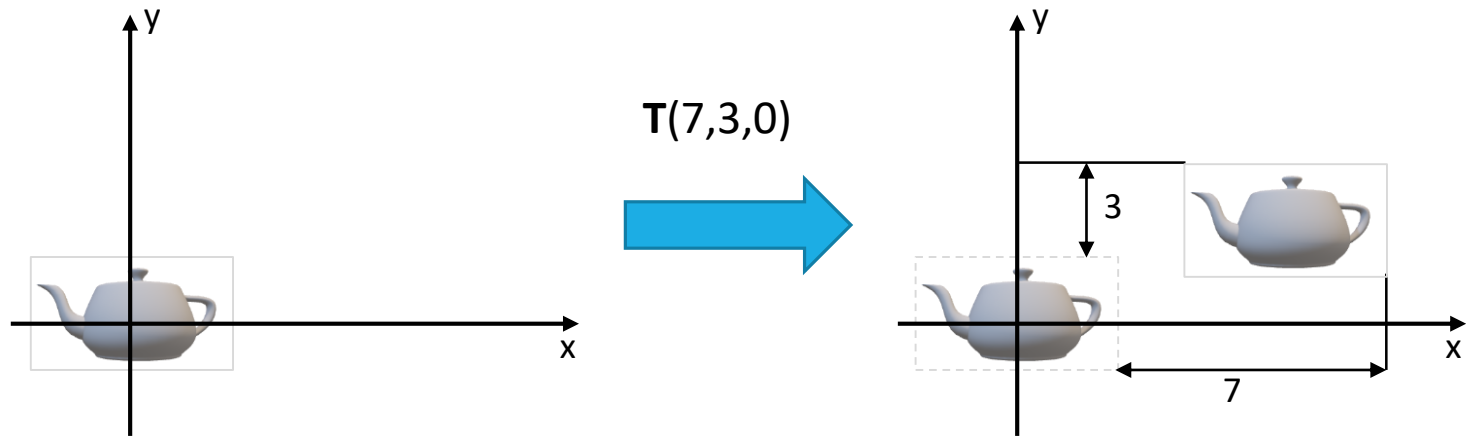
$$\mathbf{T}(\mathbf{t}) = \mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Punct $\mathbf{p} = (p_x, p_y, p_z, 1)$

Translația punctului $\mathbf{p} \rightarrow \mathbf{p}' = \mathbf{T}(\mathbf{t})\mathbf{p}$

$$\mathbf{p}' = (p_x + t_x, p_y + t_y, p_z + t_z, 1)$$

Translația



Translația vectorului $\mathbf{v} = (v_x, v_y, v_z, 0)$?

Inversa matricei de translație

$$\mathbf{T}^{-1}(\mathbf{t}) = \mathbf{T}(-\mathbf{t})$$

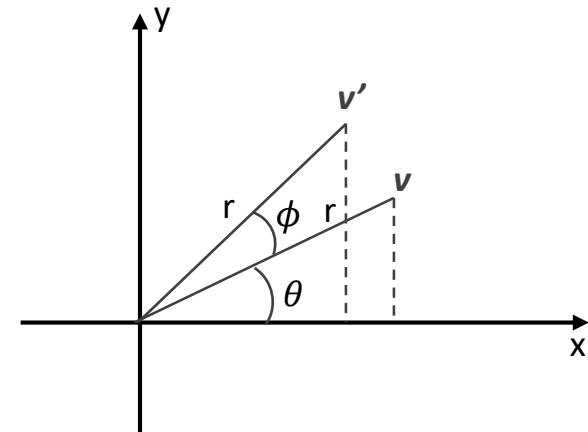
Rotația

Rotește un vector (poziție sau direcție) cu un unghi dat în jurul unei axe care trece prin origine

Punct $v = (v_x, v_y) = (r \cos\theta, r \sin\theta)$

Rotație cu ϕ grade

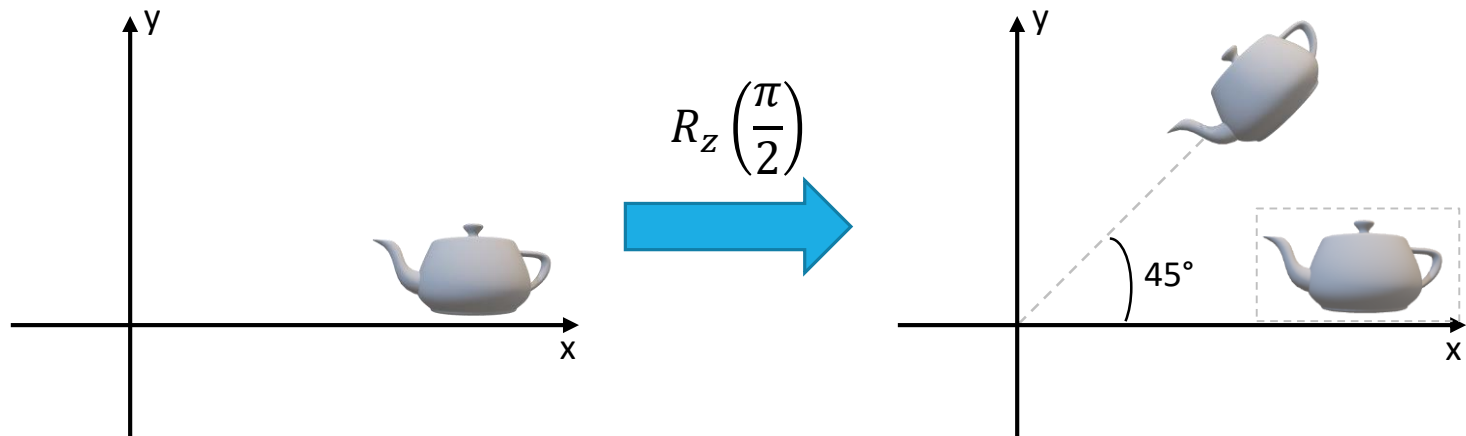
$$\begin{aligned} v' &= \begin{pmatrix} r \cos(\theta + \phi) \\ r \sin(\theta + \phi) \end{pmatrix} = \begin{pmatrix} r(\cos\theta\cos\phi - \sin\theta\sin\phi) \\ r(\sin\theta\cos\phi + \cos\theta\sin\phi) \end{pmatrix} \\ &= \underbrace{\begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}}_{\mathbf{R}(\phi)} \underbrace{\begin{pmatrix} r \cos\theta \\ r \sin\theta \end{pmatrix}}_{\mathbf{v}} = \mathbf{R}(\phi)\mathbf{v} \end{aligned}$$



În 3D:

$$\mathbf{R}_x(\phi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi & 0 \\ 0 & \sin\phi & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \mathbf{R}_y(\phi) = \begin{pmatrix} \cos\phi & 0 & \sin\phi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\phi & 0 & \cos\phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \mathbf{R}_z(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi & 0 & 0 \\ \sin\phi & \cos\phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

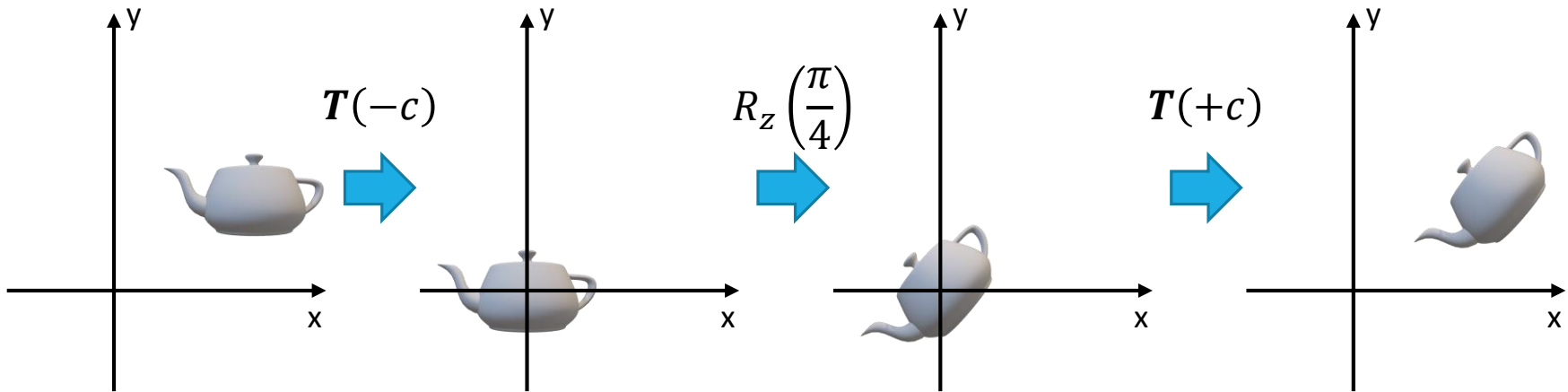
Rotația



Inversa matricei de rotație

$$R_i^{-1}(\phi) = R_i(-\phi)$$

Rotația în jurul unui punct arbitrar



$$X = T(c)R_z(\phi)T(-c)$$

Scalarea

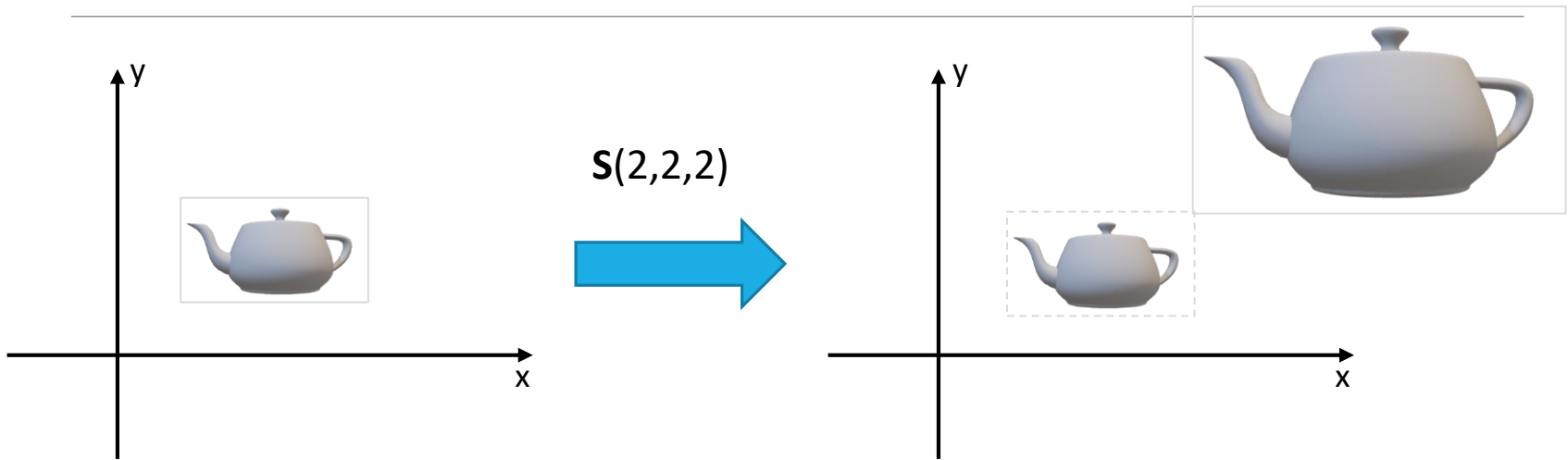
$$\mathbf{S}(\mathbf{s}) = \mathbf{S}(s_x, s_y, s_z)$$

Scalează o entitate cu factorii de scalare s_x , s_y și s_z de-a lungul axelor x , y și respectiv z

Scalare uniformă (izotropă): $s_x = s_y = s_z$, altfel – scalare non-uniformă (anizotropă)

$$\mathbf{S}(\mathbf{s}) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Scalarea



Inversa matricei de scalare

$$S^{-1}(s) = S\left(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z}\right)$$

Scalarea

Una sau trei valori negative ale componentelor lui s – matrice de reflexie

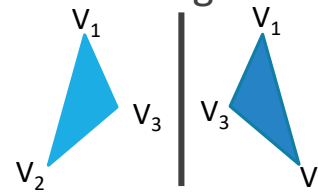
Doar două valori negative – echivalentul unei rotații cu π radiani

O matrice de rotație concatenată cu o matrice de reflexie – o matrice de reflexie

$$\underbrace{\begin{pmatrix} \cos\left(\frac{\pi}{2}\right) & \sin\left(\frac{\pi}{2}\right) \\ -\sin\left(\frac{\pi}{2}\right) & \cos\left(\frac{\pi}{2}\right) \end{pmatrix}}_{\text{rotație}} \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}}_{\text{reflexie}} = \underbrace{\begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix}}_{\text{reflexie}}$$

Matricele de reflexie trebuie tratate special când sunt întâlnite

- Ex. O matrice de reflexie modifică ordinea vârfurilor triunghiurilor (orientarea triunghiului) → probleme de iluminare



Detectarea unei matrici de reflexie

$$A = \begin{pmatrix} \overline{A} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix}$$

Matricea A reflectă dacă

$$\det(\overline{A}) < 0$$

Scalarea după o direcție arbitrară

Matricea de scalare – scalare după axele x , y și z

Scalare după alte direcții – f^x, f^y, f^z – vectori unitari perpendiculari (bază ortonormată)

Se construiește matricea de schimbare de bază

$$F = \begin{pmatrix} f^x & f^y & f^z & \mathbf{0} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matricea de scalare finală

$$X = FS(s)F^T$$

Forfecarea (shearing)

Deformarea obiectului de-a lungul unei axe în funcție de o altă axă și parametrul s

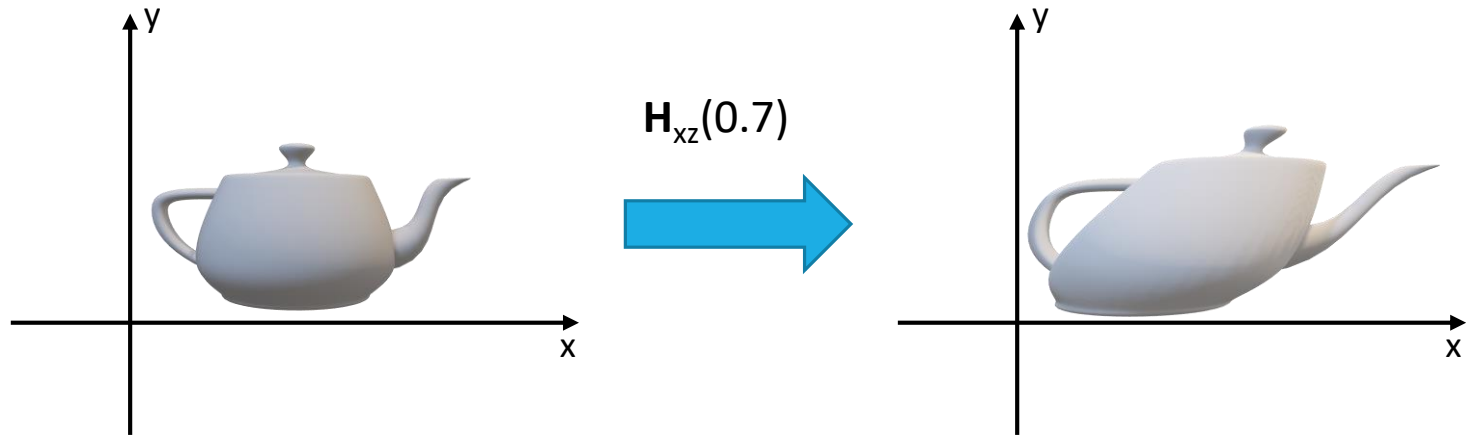
6 matrici de bază

- $H_{xy}(s), H_{xz}(s), H_{yx}(s), H_{yz}(s), H_{zx}(s), H_{zy}(s)$
- $H_{ij}(s)$ – i – coordonata care se modifică, j – coordonata în funcție de care se face forfecarea
- i – identifică linia unde se regăsește parametrul s în matrice
- j – identifică coloana unde se regăsește parametrul s în matrice

Ex. Matrice de forfecare

$$H_{xz}(s) = \begin{pmatrix} 1 & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Forfecarea (shearing)



Inversa matricei de forfecare

$$H_{ij}^{-1}(s) = H_{ij}(-s)$$

Păstrează volumul obiectelor

Concatenarea transformărilor

Concatenarea transformărilor – înmulțirea matricilor de transformare

Înmulțirea matricilor – **necomutativă**

- **Atenție la ordinea înmulțirilor!**
- Efecte diferite în funcție de ordine

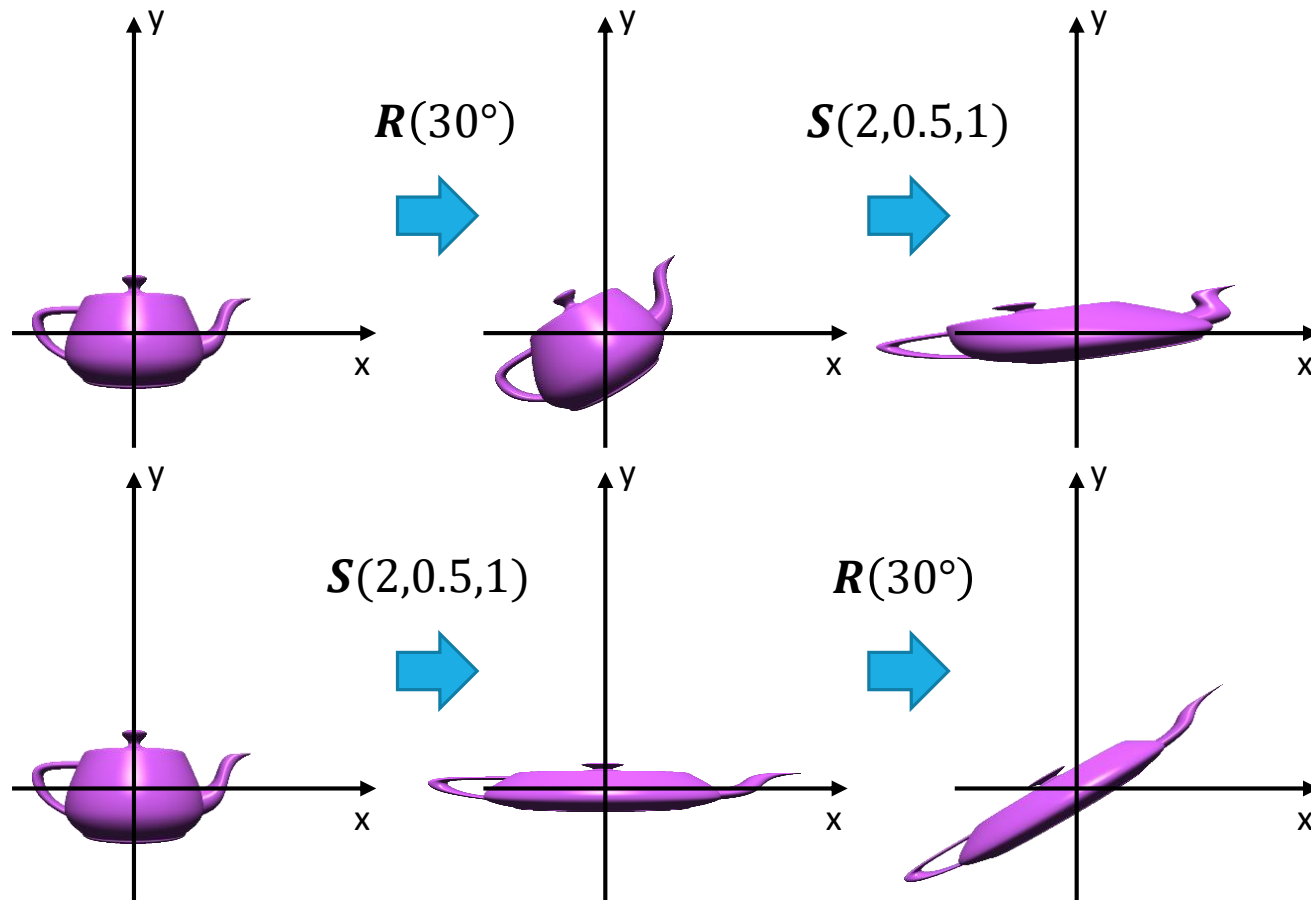
Concatenarea transformărilor – De ce?

- Eficiență
- Ex. Obiect de rezoluție foarte mare (5m vârfuri)
- Transformări aplicate obiectului – $S \rightarrow R \rightarrow T$
- Fără concatenare – 3 înmulțiri/vârf – total **15m** înmulțiri
- Cu concatenare ($C=TRS$) – 1 înmulțire/vârf – total **5m** înmulțiri

Ordinea

- $TRSp = (T(R(Sp)))$
- Asociativitate - $TRSp = (TR)(Sp)$ – ex. Calcularea componentei transformărilor rigide

Concatenarea transformărilor



Transformări rigide

Transformări care nu modifică forma obiectelor

Concatenări de translații și rotații

Păstrează lungimi, unghiuri și orientări

Matricea unei transformări rigide

$$\mathbf{X} = \mathbf{T}(\mathbf{t})\mathbf{R} = \begin{pmatrix} r_{00} & r_{01} & r_{02} & t_x \\ r_{10} & r_{11} & r_{12} & t_y \\ r_{20} & r_{21} & r_{22} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Inversa matricei unei transformări rigide

$$\mathbf{X}^{-1} = (\mathbf{T}(\mathbf{t})\mathbf{R})^{-1} = \mathbf{R}^{-1}\mathbf{T}(\mathbf{t})^{-1} = \mathbf{R}^T\mathbf{T}(-\mathbf{t})$$

Transformări rigide

Altă modalitate de a calcula inversa

Notăm

$$\bar{R} = (\mathbf{r}_{,0} \quad \mathbf{r}_{,1} \quad \mathbf{r}_{,2}) = \begin{pmatrix} \mathbf{r}_{0,}^T \\ \mathbf{r}_{1,}^T \\ \mathbf{r}_{2,}^T \end{pmatrix}$$
$$X = \begin{pmatrix} \bar{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix}$$

Atunci

$$X^{-1} = \begin{pmatrix} \mathbf{r}_{0,} & \mathbf{r}_{1,} & \mathbf{r}_{2,} & -\bar{R}^T \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Orientarea camerei

Transformare rigidă

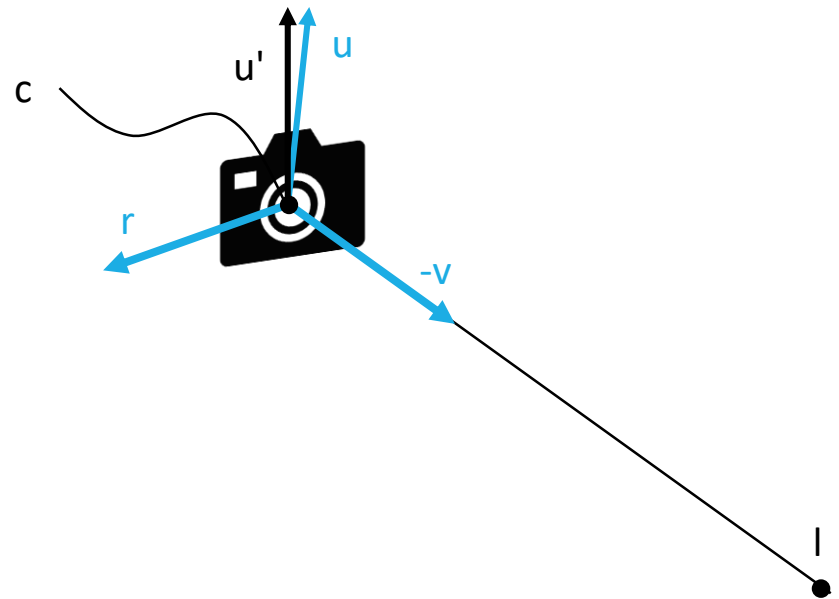
În OpenGL – funcția *gluLookAt()*

Camera

- Poziționată în c
- Privește spre l
- Direcția sus e u'

Soluția

- Calcularea bazei $\{r, u, v\}$

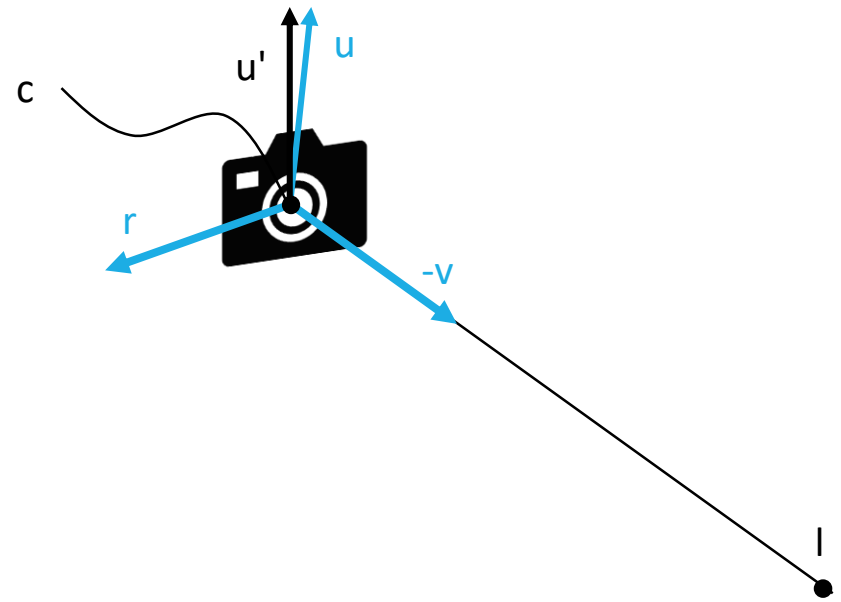


Orientarea camerei

$$\mathbf{v} = \frac{\mathbf{c} - \mathbf{l}}{\|\mathbf{c} - \mathbf{l}\|}$$

$$\mathbf{r} = -\frac{(\mathbf{v} \times \mathbf{u}')}{\|\mathbf{v} \times \mathbf{u}'\|}$$

$$\mathbf{u} = \mathbf{v} \times \mathbf{r}$$



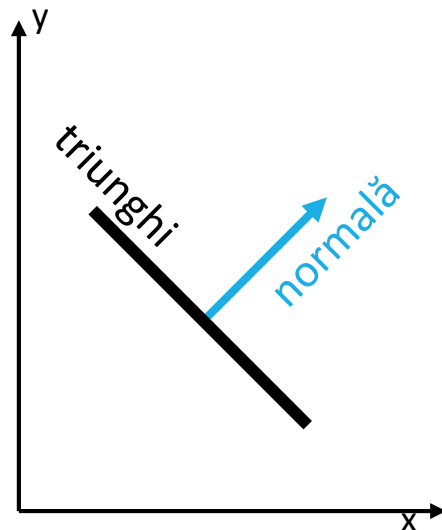
$$\mathbf{M} = \begin{pmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -t_x \\ 0 & 1 & 0 & -t_y \\ 0 & 0 & 1 & -t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{t} \cdot \mathbf{r} \\ u_x & u_y & u_z & -\mathbf{t} \cdot \mathbf{u} \\ v_x & v_y & v_z & -\mathbf{t} \cdot \mathbf{v} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Transformarea normalelor

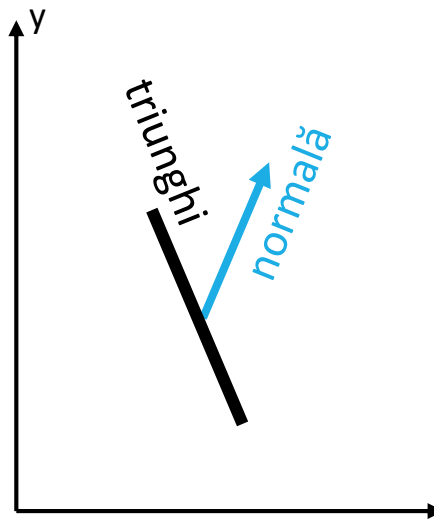
O matrice de transformări concatenate – transformarea punctelor, liniilor, triunghiurilor, vectorilor tangenți la suprafață

Nu poate fi folosită întotdeauna pentru transformarea normalelor!

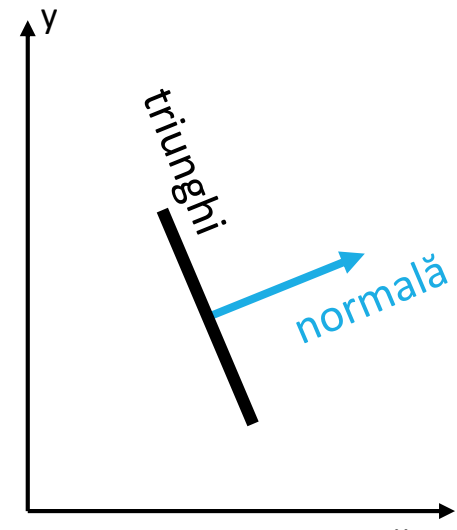
Ex.



Triunghi original



Triunghi scalat cu 0.5 pe x



Normala corectă

Transformarea normalelor

Se folosește transpusa inversei

$$(A^{-1})^T = \left(\frac{1}{\det A} \cdot A^* \right)^T$$

Foarte costisitor

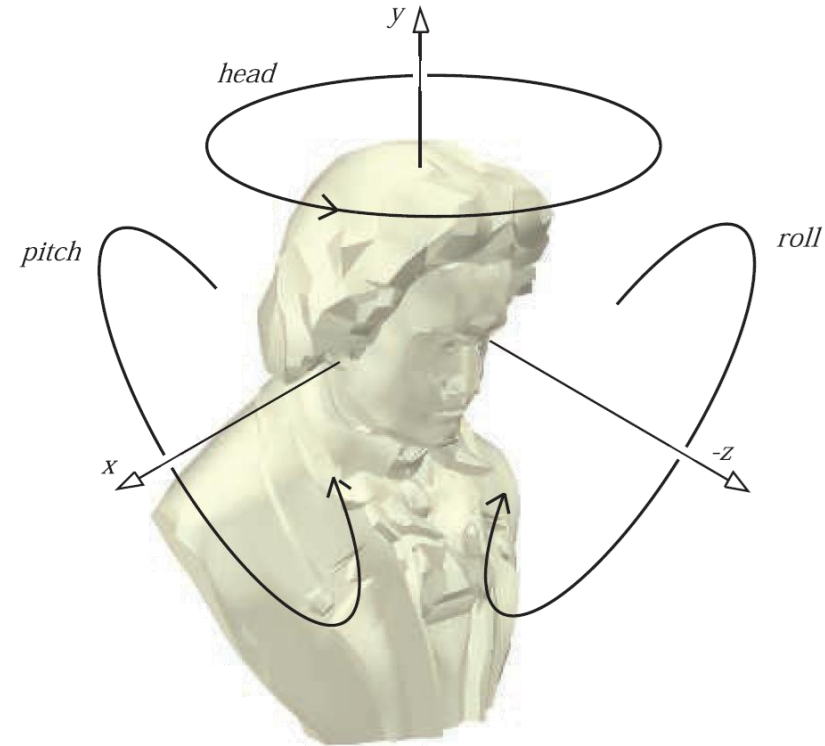
Si atunci?

$$A = \begin{pmatrix} a_{00} & a_{01} & a_{02} & t_x \\ a_{10} & a_{11} & a_{12} & t_y \\ a_{20} & a_{21} & a_{22} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \rightarrow A = \begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{pmatrix}$$

Scalări uniforme – nu afectează direcția normalelor, doar norma → pot fi ignorate

Rămân rotațiile – matrice ortogonală – $(A^{-1})^T = (A^T)^T = A$

Transformări geometrice speciale

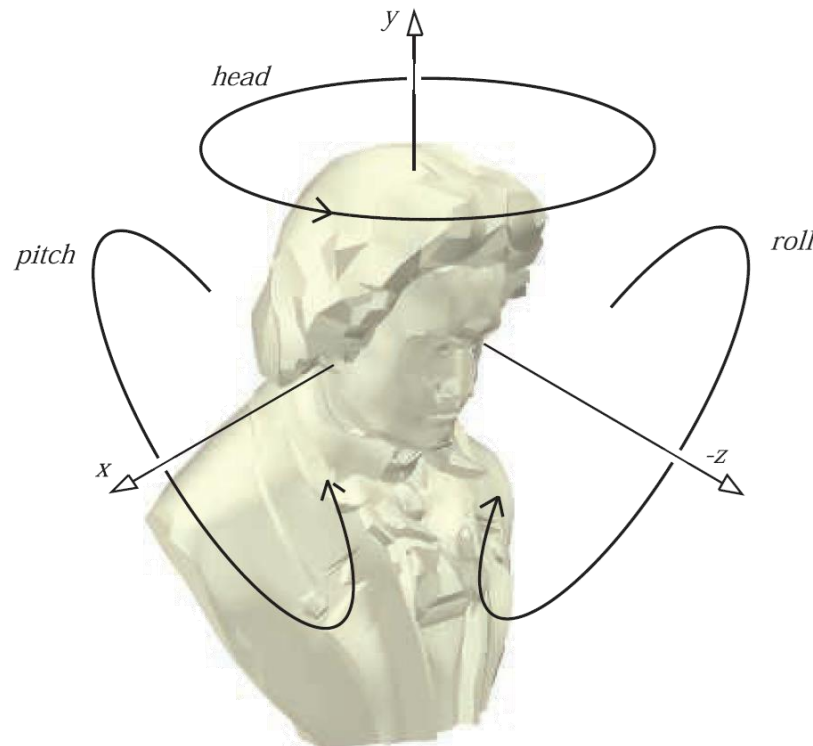
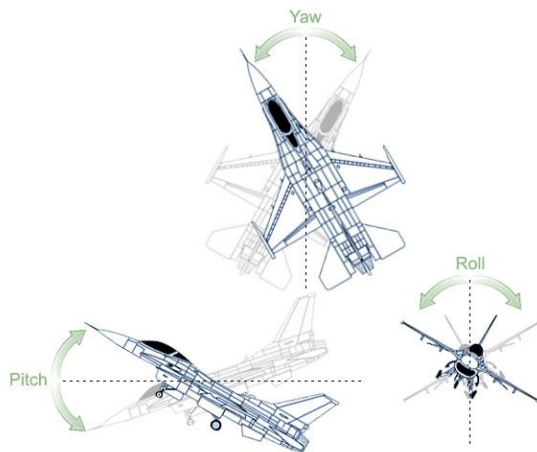


Transformarea Euler

Metoda intuitivă de a construi o matrice de orientare

Unghiurile lui Euler

- head (yaw), h
- pitch, p
- roll, r



Leonhard Euler
(1707-1783)

Sursa: Real-time Rendering (4th Edition), Tomas Akenine-Möller et al.

Extragerea parametrilor Transformării Euler

Înmulțirea a 3 matrici

$$E(h, p, r) = R_z(r)R_x(p)R_y(h) = \begin{pmatrix} e_{00} & e_{01} & e_{02} \\ e_{10} & e_{11} & e_{12} \\ e_{20} & e_{21} & e_{22} \end{pmatrix}$$

Înlocuind cele 3 matrici de rotație

$$E = \begin{pmatrix} \cos r \cos h - \sin r \sin p \sin h & -\sin r \cos p & \cos r \sin h + \sin r \sin p \cos h \\ \sin r \cos h + \cos r \sin p \sin h & \cos r \cos p & \sin r \sin h - \cos r \sin p \cos h \\ -\cos p \sin h & \sin p & \cos p \cos h \end{pmatrix}$$

$$\begin{cases} e_{21} = \sin p \\ \frac{e_{01}}{e_{11}} = \frac{-\sin r}{\cos r} = -\tan r \\ \frac{e_{20}}{e_{22}} = \frac{-\sin h}{\cos h} = -\tan h \end{cases} \rightarrow \begin{cases} h = \text{atan2}(-e_{20}, e_{22}) \\ p = \arcsin(e_{21}) \\ r = \text{atan2}(-e_{01}, e_{11}) \end{cases}$$

Gimbal lock

Caz special – $\cos p$
– *Gimbal lock*

Se pierde un grad
de libertate –
celelalte două
rotații se fac după
aceeași axă



EULER EXPLAINED

Euler (gimbal lock) Explained – <https://www.youtube.com/watch?v=zc8b2Jo7mno>

Gimbal lock

h și r rotesc în jurul aceleiași axe $\rightarrow$ putem considera arbitrar una dintre valori zero

- Ex. $h = 0$

$$E = \begin{pmatrix} \cos r & \sin r \cos p & \sin r \sin p \\ \sin r & \cos r \cos p & -\cos r \sin p \\ 0 & \sin p & \cos p \end{pmatrix}$$

Cum p nu afectează valorile din prima coloană

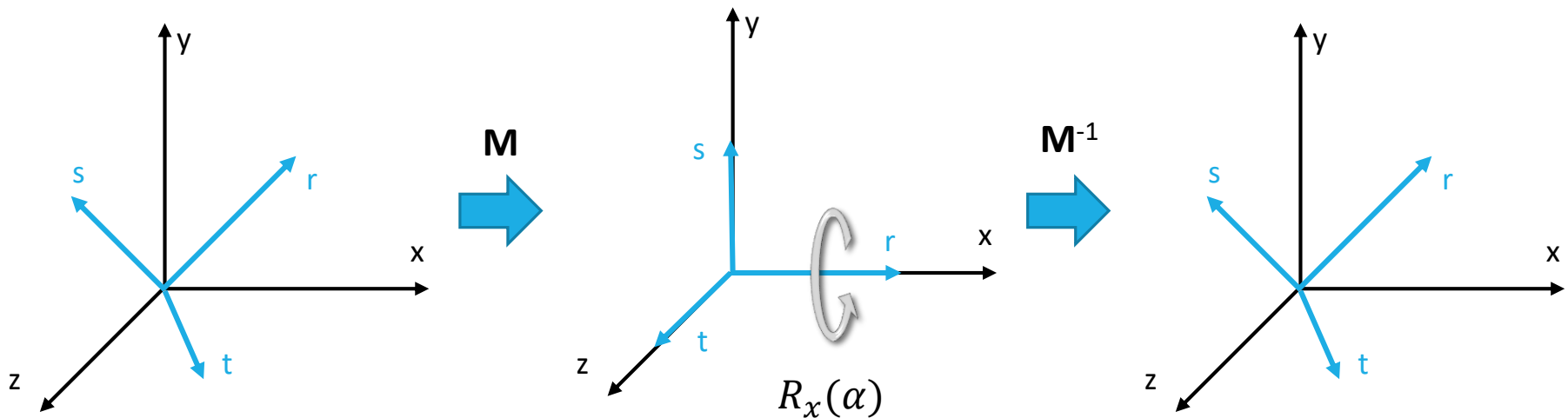
$$\frac{e_{10}}{e_{00}} = \frac{\sin r}{\cos r} = \tan r \rightarrow r = \text{atan2}(e_{10}, e_{00})$$

Rotația în jurul unei axe arbitrare

Axă arbitrară $\mathbf{r}$, normalizată

Generarea unei transformări care rotește în jurul axei $\mathbf{r}$ cu α radiani

Generarea unei matrici $\mathbf{M}$ care transformă într-un spațiu în care axa $\mathbf{r}$ coincide cu axa x



Rotația în jurul unei axe arbitrare

Găsim axa $\mathbf{s}$ și calculăm axa $\mathbf{t}$ ca $\mathbf{t} = \mathbf{r} \times \mathbf{s}$

Pentru a calcula $\mathbf{s}$

- Identificăm cea mai mică componentă (în modul) a lui $\mathbf{r}$ și o facem 0
- Interschimbăm celelalte două componente și o negăm pe prima dintre ele

$$\bar{\mathbf{s}} = \begin{cases} (0, -r_z, r_y), \text{ dacă } |r_x| \leq |r_y| \text{ și } |r_x| \leq |r_z|, \\ (-r_z, 0, r_x), \text{ dacă } |r_y| \leq |r_x| \text{ și } |r_y| \leq |r_z|, \\ (-r_y, r_x, 0), \text{ dacă } |r_z| \leq |r_x| \text{ și } |r_z| \leq |r_y|, \end{cases}$$

$$\mathbf{s} = \frac{\bar{\mathbf{s}}}{\|\bar{\mathbf{s}}\|} \quad \mathbf{M} = \begin{pmatrix} \mathbf{r}^T \\ \mathbf{s}^T \\ \mathbf{t}^T \end{pmatrix}$$

$$\mathbf{t} = \mathbf{r} \times \mathbf{s}$$

$$\mathbf{X} = \mathbf{M}^T \mathbf{R}_x(\alpha) \mathbf{M}$$

Transformarea Euler - Camera



Euler angles camera - <https://learnopengl.com/Getting-started/Camera>



Cuaternioni

Cuaternioni

Invența de matematicianul William Rowan Hamilton
în 1843

Extensie a numerelor complexe

1985 – introduși în grafică

Utilizați pentru a reprezenta rotații și orientări

Superiori transformărilor Euler sau matriceale

Pot fi utilizați pentru interpolare constantă și stabilă a
orientărilor



Sir William
Rowan Hamilton
(1805–1865)

Cuaternioni

Definiție

$$\begin{aligned}\hat{q} &= (\mathbf{q}_v, q_w) = iq_x + jq_y + kq_z + q_w = \mathbf{q}_v + q_w, \\ \mathbf{q}_v &= iq_x + jq_y + kq_z = (q_x, q_y, q_z) \\ i^2 &= j^2 = k^2 = -1, jk = -kj = i, ki = -ik = j, ij = -ji = k.\end{aligned}$$

Parte reală – q_w , parte imaginară – $\mathbf{q}_v$, și i, j, k – unități imaginare

Părțile imaginare – $\mathbf{q}_v$ i se pot aplica toate operațiile vectoriale

- Ex. adunare, scalare, produs scalar, produs vectorial etc.

Cuaternioni – Proprietăți matematice

Înmulțirea

$$\begin{aligned}\hat{\mathbf{q}}\hat{\mathbf{r}} &= (iq_x + jq_y + kq_z + q_w)(ir_x + jr_y + kr_z + r_w) \\ &= i(q_yr_z - q_zr_y + r_wq_x + q_wr_x) \\ &\quad + j(q_zr_x - q_xr_z + r_wq_y + q_wr_y) \\ &\quad + k(q_xr_y - q_yr_x + r_wq_z + q_wr_z) \\ &\quad + q_wr_w - q_xr_x - q_yr_y - q_zr_z \\ &= (\mathbf{q}_v \times \mathbf{r}_v + r_w\mathbf{q}_v + q_w\mathbf{r}_v, q_wr_w - \mathbf{q}_v \cdot \mathbf{r}_v).\end{aligned}$$

Adunarea

$$\hat{\mathbf{q}} + \hat{\mathbf{r}} = (\mathbf{q}_v, q_w) + (\mathbf{r}_v, r_w) = (\mathbf{q}_v + \mathbf{r}_v, q_w + r_w)$$

Cuaternioni – Proprietăți matematice

Conjugat

$$\hat{q}^* = (q_v, q_w)^* = (-q_v, q_w)$$

Norma

$$n(\hat{q}) = \sqrt{\hat{q}\hat{q}^*} = \sqrt{\hat{q}^*\hat{q}} = \sqrt{q_v \cdot q_v + q_w^2} = \sqrt{q_x^2 + q_y^2 + q_z^2 + q_w^2}$$

Inversa

$$\begin{aligned}\hat{q}^{-1}\hat{q} &= \hat{q}\hat{q}^{-1} = 1 \\ n(\hat{q})^2 &= \hat{q}\hat{q}^* \Leftrightarrow \frac{\hat{q}\hat{q}^*}{n(\hat{q})^2} = 1 \\ \hat{q}^{-1} &= \frac{1}{n(\hat{q})^2} \hat{q}^*\end{aligned}$$

Cuaternioni unitate

$\hat{q}(q_v, q_w)$ – cuaternion unitate $\Leftrightarrow n(\hat{q}) = 1$

Putem rescrie

$$\hat{q} = (\sin \phi \mathbf{u}_q, \cos \phi) = \sin \phi \mathbf{u}_q + \cos \phi, \text{ unde } \|\mathbf{u}_q\| = 1$$

Întrucât

$$\begin{aligned} n(\hat{q}) &= n(\sin \phi \mathbf{u}_q, \cos \phi) = \sqrt{\sin^2 \phi (\mathbf{u}_q \cdot \mathbf{u}_q) + \cos^2 \phi} \\ &= \sqrt{\sin^2 \phi + \cos^2 \phi} = 1 \end{aligned}$$

Forma logaritmică

$$\hat{q} = \sin \phi \mathbf{u}_q + \cos \phi = e^{\phi \mathbf{u}_q}$$

$$\log(\hat{q}) = \log(e^{\phi \mathbf{u}_q}) = \phi \mathbf{u}_q$$

$$\hat{q}^t = (\sin \phi \mathbf{u}_q + \cos \phi)^t = e^{\phi t \mathbf{u}_q} = \sin(\phi t) \mathbf{u}_q + \cos(\phi t)$$

Transformări cu cuaternioni

Punct $\mathbf{p} = (p_x, p_y, p_z, p_w)^T \rightarrow$ cuaternion $\hat{\mathbf{p}}$

$\hat{\mathbf{q}} = (\sin \phi \mathbf{u}_q, \cos \phi)$ – cuaternion unitate

Rotația cuaternionului $\hat{\mathbf{p}}$ (implicit a punctului $\mathbf{p}$) după axa $\mathbf{u}_q$ cu 2ϕ

$$\hat{\mathbf{q}}\hat{\mathbf{p}}\hat{\mathbf{q}}^{-1}$$

Cuaternioni unitate $\rightarrow \hat{\mathbf{q}}^{-1} = \hat{\mathbf{q}}^*$

Concatenarea rotațiilor

$$\hat{\mathbf{r}}(\hat{\mathbf{q}}\hat{\mathbf{p}}\hat{\mathbf{q}}^*)\hat{\mathbf{r}}^* = (\hat{\mathbf{r}}\hat{\mathbf{q}})\hat{\mathbf{p}}(\hat{\mathbf{r}}\hat{\mathbf{q}})^* = \hat{\mathbf{c}}\hat{\mathbf{p}}\hat{\mathbf{c}}^*, \\ \hat{\mathbf{c}} = \hat{\mathbf{r}}\hat{\mathbf{q}}$$

Conversia matriceală

Un cuaternion $\hat{q}$ poate fi convertit într-o matrice M^q

$$M^q = \begin{pmatrix} 1 - s(q_y^2 + q_z^2) & s(q_x q_y - q_w q_z) & s(q_x q_z + q_w q_y) & 0 \\ s(q_x q_y + q_w q_z) & 1 - s(q_x^2 + q_z^2) & s(q_y q_z - q_w q_x) & 0 \\ s(q_x q_z - q_w q_y) & s(q_y q_z + q_w q_x) & 1 - s(q_x^2 + q_y^2) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Unde $s = 2/(n(\hat{q}))^2$

Pentru cuaternioni unitate

$$M^q = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) & 0 \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) & 0 \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Conversia matriceală inversă

$$\mathbf{M}^q \rightarrow \hat{\mathbf{q}}$$

$$m_{21}^q - m_{12}^q = 4q_w q_x$$

$$m_{02}^q - m_{20}^q = 4q_w q_y$$

$$m_{10}^q - m_{01}^q = 4q_w q_z$$

$$\begin{aligned} \text{tr}(\mathbf{M}^q) &= 4 - 2s(q_x^2 + q_y^2 + q_z^2) = 4 \left(1 - \frac{q_x^2 + q_y^2 + q_z^2}{q_x^2 + q_y^2 + q_z^2 + q_w^2} \right) \\ &= \frac{4q_w^2}{q_x^2 + q_y^2 + q_z^2 + q_w^2} = \frac{4q_w^2}{(n(\hat{\mathbf{q}}))^2} \end{aligned}$$

Pentru cuaternioni unitate

$$q_w = \frac{1}{2} \sqrt{\text{tr}(\mathbf{M}^q)}$$

$$q_y = \frac{m_{02}^q - m_{20}^q}{4q_w}$$

$$q_x = \frac{m_{21}^q - m_{12}^q}{4q_w}$$

$$q_z = \frac{m_{10}^q - m_{01}^q}{4q_w}$$

Interpolare liniară sferică

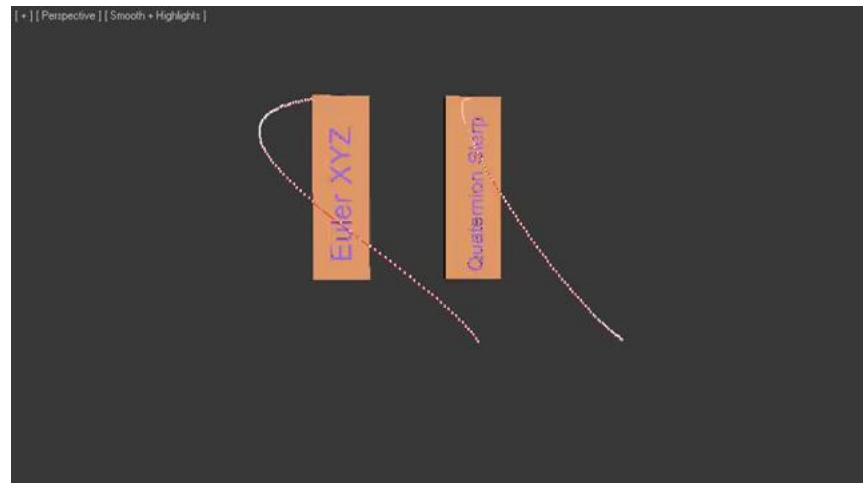
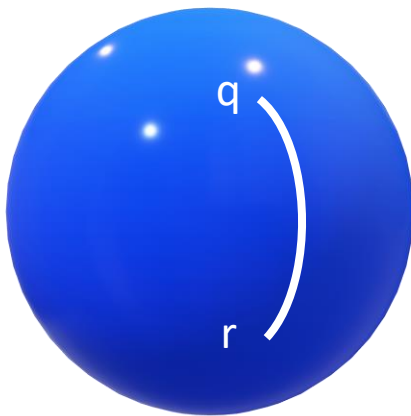
$\hat{q}$ și $\hat{r}$ - cuaternioni unitate

Parametrul $t \in [0,1]$

Cuaternionul interpolat $\hat{s}$ (cel mai scurt arc pe o sferă 4D de la $\hat{q}(t=0)$ la $\hat{r}(t=1)$)

$$\hat{s}(\hat{q}, \hat{r}, t) = \text{slerp}((\hat{q}, \hat{r}, t) = \frac{\sin(\phi(1-t))}{\sin \phi} \hat{q} + \frac{\sin(\phi t)}{\sin \phi} \hat{r}$$

$$\cos \phi = q_x r_x + q_y r_y + q_z r_z + q_w r_w$$



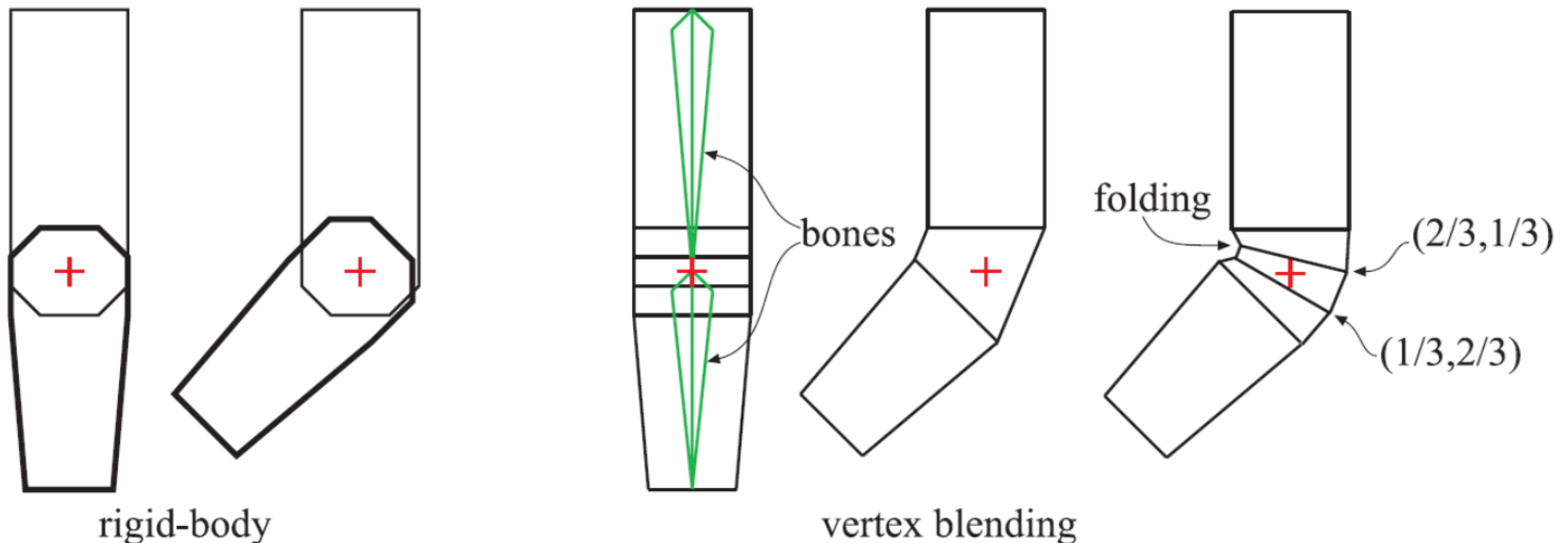
Slerp vs Euler – <https://www.youtube.com/watch?v=QxldIZ0eKCE>

Vertex blending

Tehnică utilizată pentru animarea încheieturilor flexibile

Superioară transformărilor rigide

Presupune interconectarea porțiunilor fixe (atașate unor oase) cu porțiuni deformabile (înveliș deformabil - „skin”)



Sursa: Real-time Rendering (4th Edition), Tomas Akenine-Möller et al.

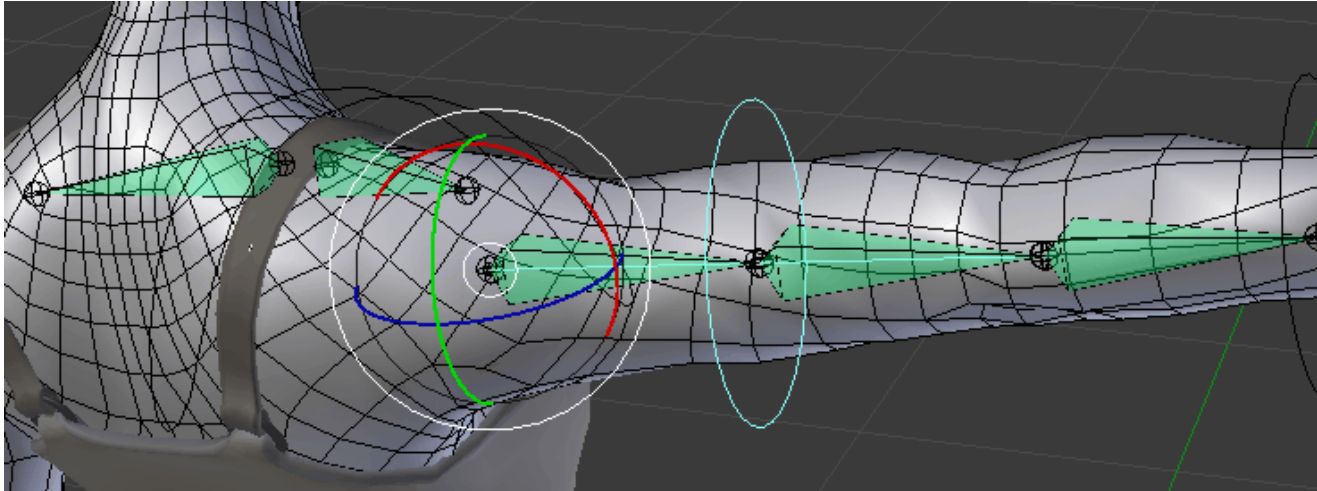
Vertex blending

Presupune transformarea triunghiurilor (vârfurilor) porțiunii flexibile cu mai multe matrici model – matricile model ale oaselor adiacente

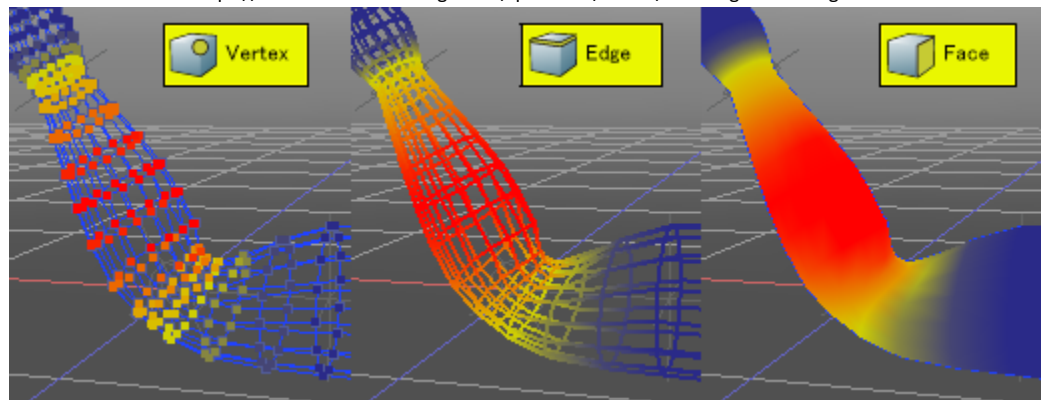
$$u(t) = \sum_{i=0}^{n-1} w_i \mathbf{B}_i(t) \mathbf{M}_i^{-1} \mathbf{p}, \text{ unde } \sum_{i=0}^{n-1} w_i = 1, w_i \geq 0$$

n - numărul de oase care influențează poziția lui $\mathbf{p}$, w_i – ponderea osului i pentru vârful $\mathbf{p}$, $\mathbf{M}_i$ – matricea care transformă din sistemul de coordonate al osului i în spațiul scenă, $\mathbf{B}_i(t)$ – matricea pentru transformarea în spațiul scenă a osului i , care se modifică în timp pentru a anima obiectul

Vertex blending



<https://blender.stackexchange.com/questions/41695/blending-bone-weights>



http://help.shade3d.jp/en/support/ShadeHelp/latest/Manual/Reference/200_Animation_Related/WeightPainting/02_WeightPainting.html

Morfism

Transformarea unui model geometric în altul prin interpolare în timp ($t_0 \rightarrow t_1$)

Două probleme majore

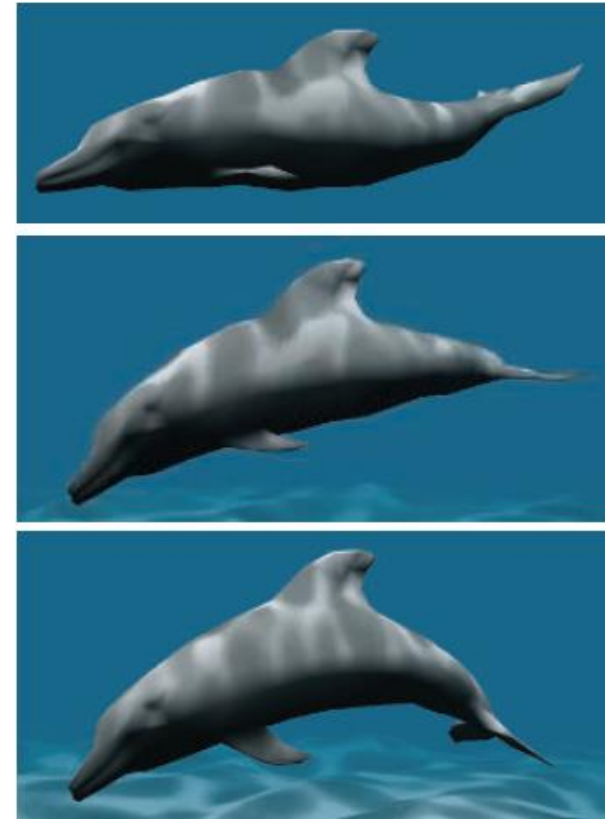
- Corespondența vârfurilor
- Problema interpolării

Dacă există corespondență 1 – 1 între vârfuri – interpolare liniară

$$s = (t - t_0) / (t_1 - t_0)$$

$$\mathbf{m} = (1 - s)\mathbf{p}_0 + s\mathbf{p}_1$$

unde $\mathbf{p}_0$ și $\mathbf{p}_1$ reprezintă vârful $\mathbf{p}$ la momentele de timp t_0 respectiv t_1



Vertex morphing
(NVIDIA)

Morph targets (Blend shapes)

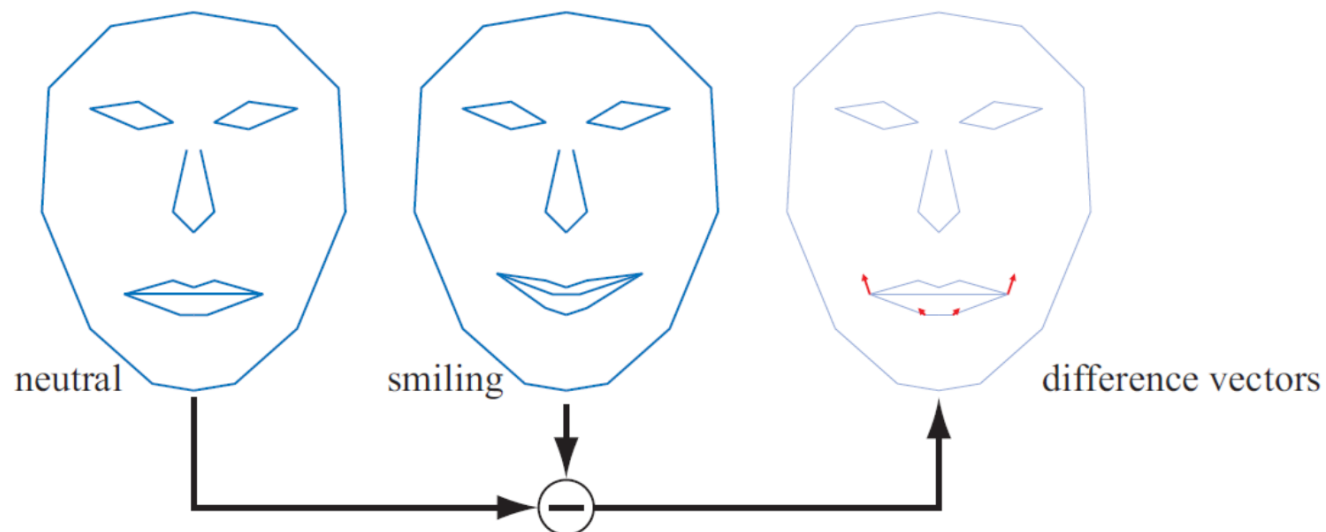
Variantă de morfism

Model neutru $\mathcal{N}$, mulțime de ipostaze $\mathcal{P}_i, i \in [1, \dots, k]$

$\mathcal{D}_i = \mathcal{P}_i - \mathcal{N}$, mulțimile diferență

Un modelul transformat (prin morfism)

$$\mathcal{M} = \mathcal{N} + \sum_{i=1}^k w_i \mathcal{D}_i$$

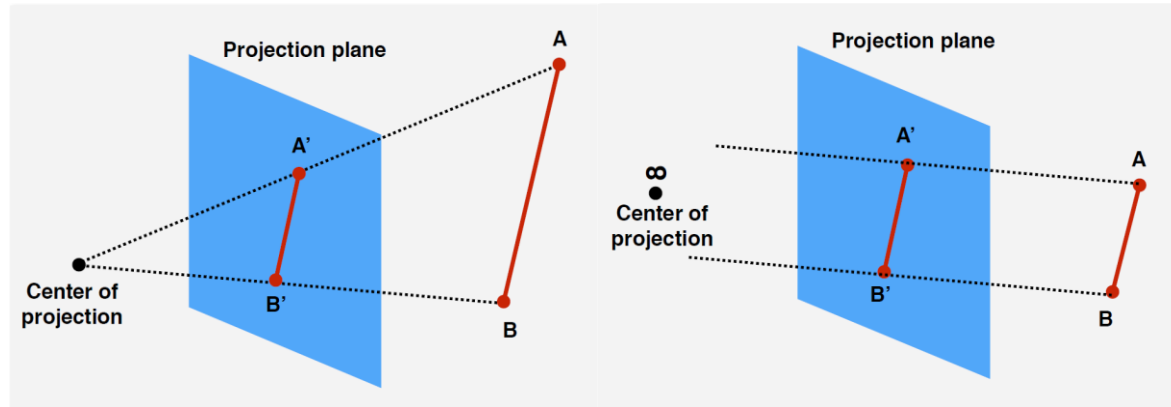


Sursa: Real-time Rendering (4th Edition), Tomas Akenine-Möller et al.

Skinning și morfism



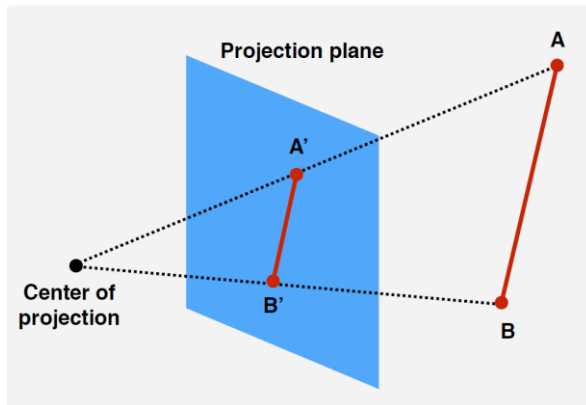
Skinning și morfism pentru animarea personajelor
(inFAMOUS Second Son)



Proiecții

Proiecții

PERSPECTIVĂ



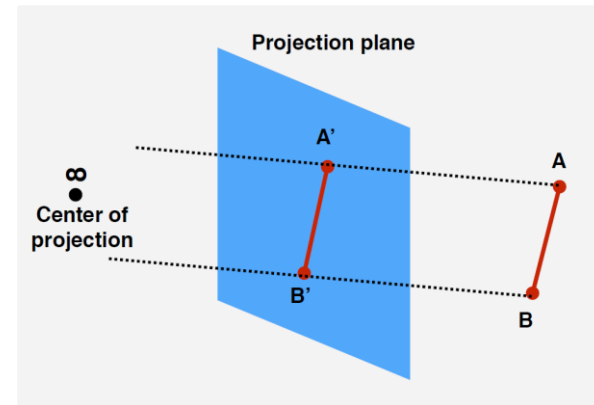
Razele de proiecție converg către centrul de proiecție (locația privitorului)

Obiectele par mai mici odată cu adâncimea

Liniile paralele cu planul de proiecție rămân paralele

Liniile care nu sunt paralele cu planul de proiecție converg către un singur punct

PARALELĂ



Raze de proiecție paralele

Punctul de convergență – la infinit

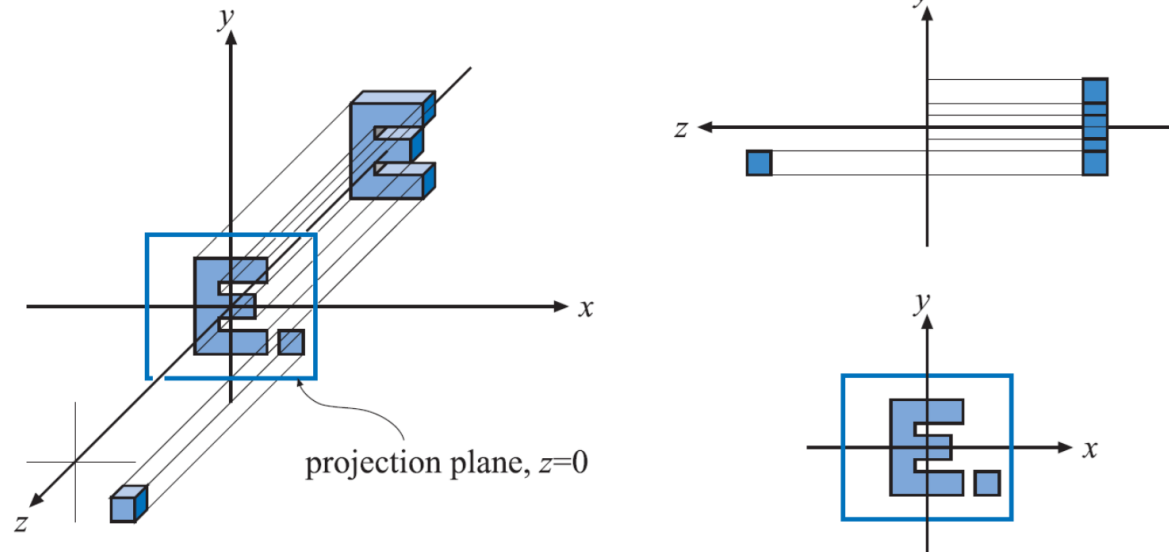
Poziția privitorului – la infinit

Liniile paralele rămân paralele

Proiecția ortogonală

Componentele x și y rămân neschimbate, componenta z devine zero (proiecție pe planul $z=0$)

$$P_o = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



Sursa: Real-time Rendering (4th Edition), Tomas Akenine-Möller et al.

Proiecția ortogonală

P_o – proiectează atât punctele cu z pozitiv cât și cele cu z negativ

$|P_o| = 0 \Rightarrow P_o$ - nu este inversabilă (coordonata z originală nu poate fi recuperată)

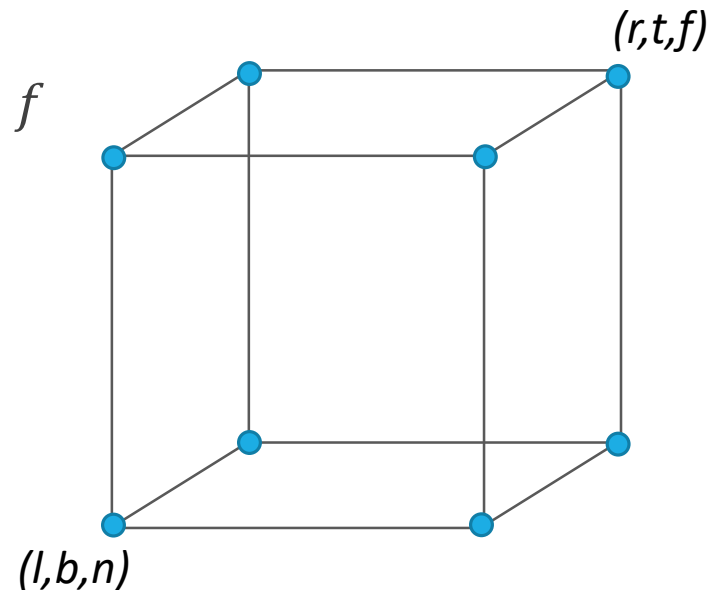
Restricționarea coordonatelor care vor fi proiectate – volum de vizualizare

$$(l, r, b, t, n, f), n > f$$

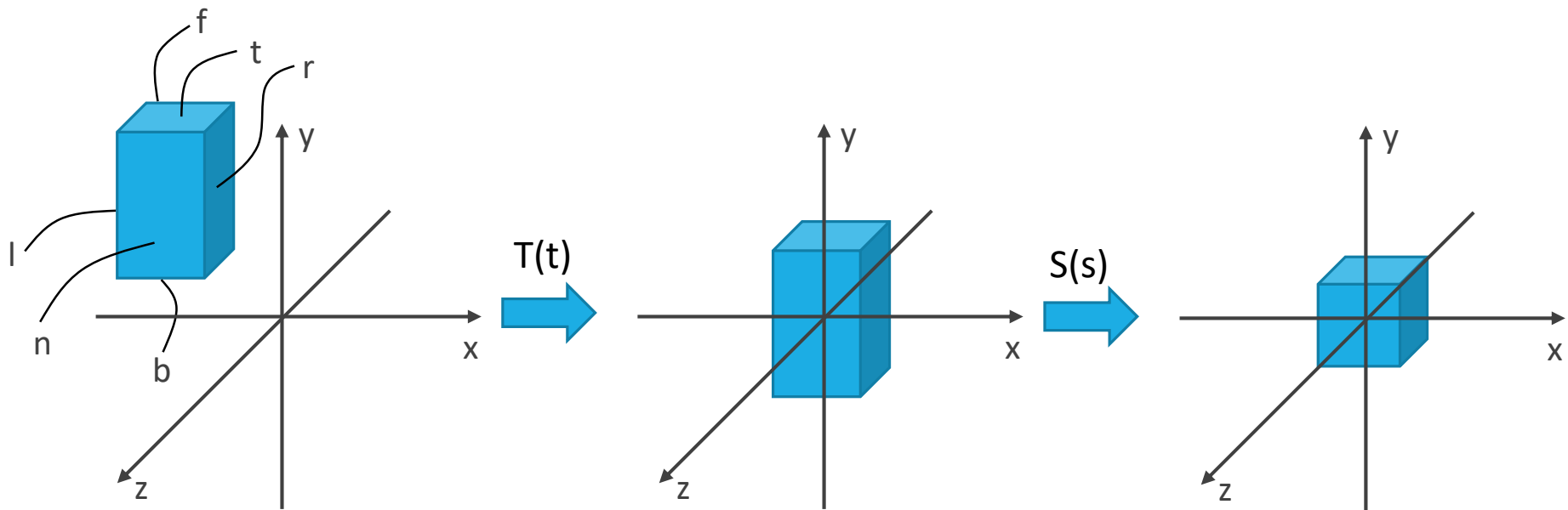
Transformarea volumului de vizualizare (AABB – axis aligned bounding box) într-un cub aliniat cu axele și centrat în origine (cub unitate)

OpenGL – $(-1, -1, -1) \rightarrow (1, 1, 1)$

DirectX – $(-1, -1, 0) \rightarrow (1, 1, 1)$



Proiecția ortogonală



Proiecția ortogonală

$$\begin{aligned}
 P_o = S(s)T(t) &= \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -\frac{l+r}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{2} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{2} \\ 0 & 0 & \frac{2}{f-n} & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}
 \end{aligned}$$

Proiecția ortogonală

P_o – inversabilă – $P_o^{-1} = T(-t)S(1/s)$

După proiecție (de obicei) se folosește un sistem de coordonate *left handed* → oglindire pe axa z

$$P_o = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matricea de proiecție ortogonală în OpenGL

$$P_o = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{l+r}{2} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{2} \\ 0 & 0 & \frac{2}{n-f} & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Proiecția perspectivă

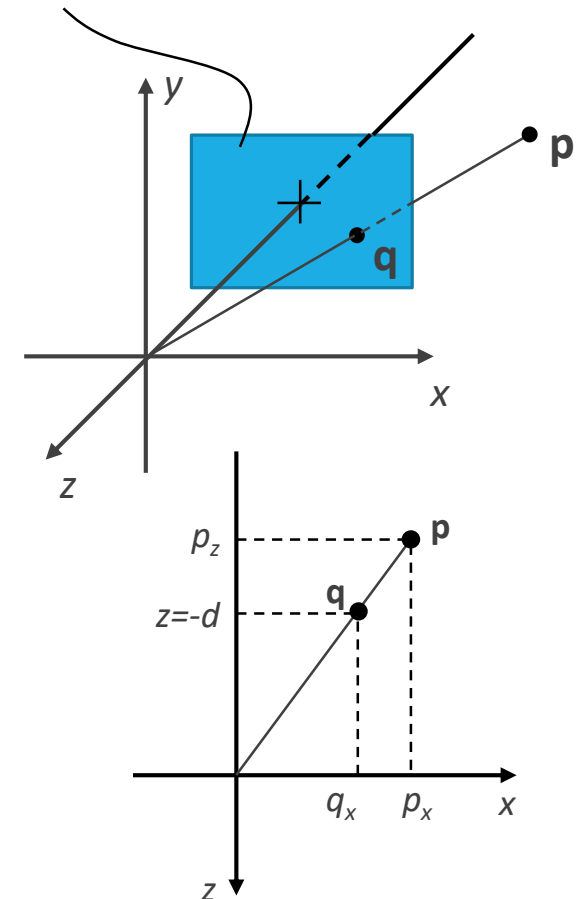
Camera centrată în origine, proiecția unui punct p pe planul $z = -d, d > 0$ va fi punctul $q(q_x, q_y, -d)$

$$\frac{q_x}{p_x} = \frac{-d}{p_z} \Leftrightarrow q_x = -d \frac{p_x}{p_z}$$
$$\frac{q_y}{p_y} = \frac{-d}{p_z} \Leftrightarrow q_y = -d \frac{p_y}{p_z}$$

Matricea perspectivă

$$P_p = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{pmatrix}$$

Planul de proiecție, $z=-d$



Proiecția perspectivă

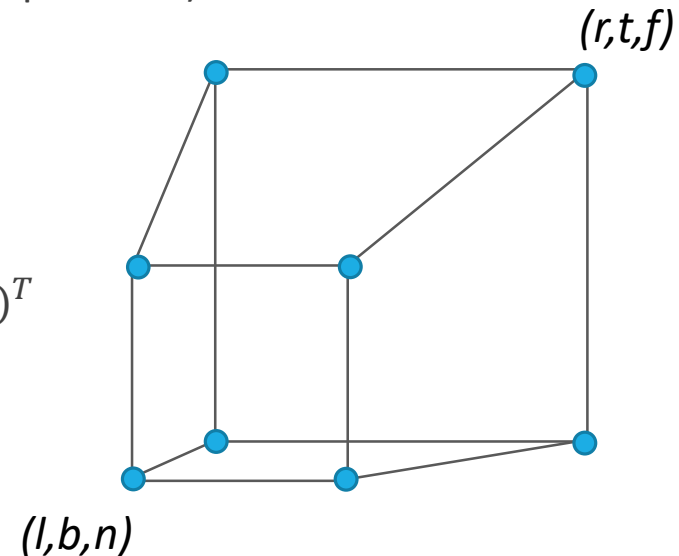
$$q = P_p p = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} = \begin{pmatrix} p_x \\ p_y \\ p_z \\ -p_z/d \end{pmatrix} \Rightarrow \begin{pmatrix} -dp_x/p_z \\ -dp_y/p_z \\ -d \\ 1 \end{pmatrix}$$

Alinierea volumului de vizualizare (frustrum – trunchi de piramidă) cu cubul unitate
 $(l, r, t, b, n, f), 0 > n > f$

Matricea perspectivă în OpenGL

$$P_p = \begin{pmatrix} \frac{2n}{r-l} & 0 & -\frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & -\frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}, q = (q_x, q_y, q_z, q_w)^T$$

$$p = (q_x/q_w, q_y/q_w, q_z/q_w, 1)$$



Proiecția perspectivă

Încorporând reflecția pe z

$$\mathbf{P}_p = \begin{pmatrix} \frac{2n'}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n'}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f'+n'}{f'-n'} & -\frac{2f'n'}{f'-n'} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

sau

$$\mathbf{P}_p = \begin{pmatrix} c/a & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & -\frac{f'+n'}{f'-n'} & -\frac{2f'n'}{f'-n'} \\ 0 & 0 & -1 & 0 \end{pmatrix}, c = 1.0/\tan(\phi/2),$$

ϕ – câmpul vizual, a – raportul de vizualizare (aspect ratio)

Proiecția perspectivă

Câmpul vizual

- Aproximativ 60° în jocurile pe console
- Aproximativ $90^\circ - 100^\circ$ în jocurile pe PC

FOV: 73.15°



FOV: 106.15°



[http://pyroteq.com/wp-content/uploads/2014/07/farcry_3_fov_comparison.jpg]

Transformările în OpenGL



Transformările în OpenGL

Se aplică la nivel de shader (vertex shader)

Matricile de transformare – generate și transmise din aplicație

Generarea matricelor de transformare – librăria GLM

Ex.

Translația

- `glm::mat4 m = glm::translate(glm::mat4(1.0f), glm::vec3(1.0f));`

Scalarea

- `glm::mat4 m = glm::scale(glm::mat4(1.0f), glm::vec3(2.0f));`

Rotația

- `glm::mat4 m = glm::rotate(glm::mat4(1.0f), glm::radians(45.0f), glm::vec3(1.0f, 0.0f, 0.0f));`

Transformările în OpenGL

Matricea de orientare a camerei

- `glm::mat4 m = glm::lookAt(glm::vec3(0.0f), glm::vec3(0.0f, 0.0f, -1.0f), glm::vec3(0.0f, 1.0f, 0.0f));`

Matricea de rotație din unghiurile Euler

- `glm::mat4 m = glm::yawPitchRoll(yaw, pitch, roll);`

Cuaternioni

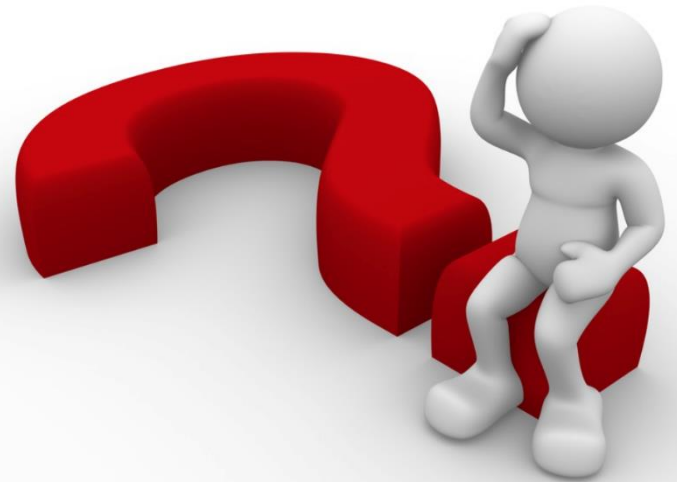
- `glm::quat q;`
- `glm::quat q = glm::quat(glm::vec3(yaw,pitch,roll));` //cuaternion din unghiuri Euler
- `glm::mat4 m = glm::mat4_cast(q);` //cuaternion în mat4

Matricea proiecție

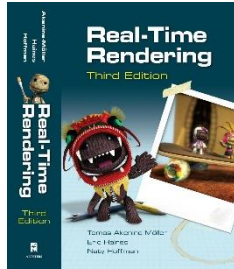
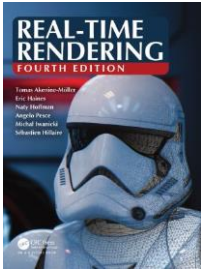
- ortogonală: `glm::mat4 m = glm::ortho(left, right, bottom, top, near, far);`
- perspectivă: `glm::mat4 m = glm::perspective(fovY, aspect, near, far);`

Întrebări

ASKING QUESTIONS

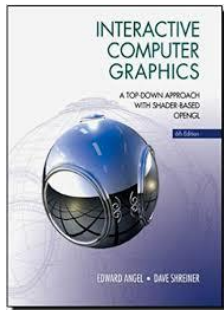


Bibliografie



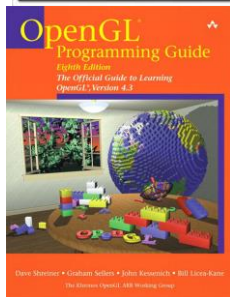
Real-time Rendering (4th Edition sau 3rd Edition)

- Capitolul 4 – Transforms



Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL (6th Edition)

- Capitolul 3 – Geometric Objects and Transformations
- Capitolul 4 – Viewing



OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th Edition)

- Capitolul 5 – Viewing Transformations, Clipping, and Feedback